Transformers
Part 1

Slides mostly borrowed from materials by Jay Alammar, Pawan Goyal,
the Stanford NLP course materials and other online resources

Transformer architecture introduced in 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Great Results with Transformers: Neural MT

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

[Test sets: WMT 2014 English-German and English-French]

Transformers able to perform better than all prior architectures

Cost of training Transformers is smaller than that for most prior architectures

https://web.stanford.edu/class/cs224n/

Great Results with Transformers: Rise of LLMs

Today, Transformer-based models dominate LMSYS Chatbot Arena Leaderboard!

Rank	Model	Arena Elo	95% CI	Votes	Organization	License	Knowledge Cutoff
1	GPT-4-Turbo-2024-04-09	1258	+4/-4	26444	OpenAI	Proprietary	2023/12
1	GPT-4-1106-preview	1253	+3/-3	68353	OpenAI	Proprietary	2023/4
1	Claude 3 Opus	1251	+3/-3	71500	Anthropic	Proprietary	2023/8
2	Gemini 1.5 Pro API-0409-Preview	1249	+4/-5	22211	Google	Proprietary	2023/11
3	GPT-4-0125-preview	1248	+2/-3	58959	OpenAI	Proprietary	2023/12
6	Meta Llama 3 70b Instruct	1213	+4/-6	15809	Meta	Llama 3 Community	2023/12
6	Bard (Gemini Pro)	1208	+7/-6	12435	Google	Proprietary	Online
7	Claude 3 Sonnet	1201	+4/-2	73414	Anthropic	Proprietary	2023/8

Gemini / Bard (Google)

ChatGPT / GPT-4 (OpenAI)

Claude 3 (Anthropic)

Llama 3 (Meta)

[Chiang et al., 2024]

https://web.stanford.edu/class/cs224n/

Transformers have shown promise outside NLP

Protein Folding

[Jumper et al. 2021] aka AlphaFold2!

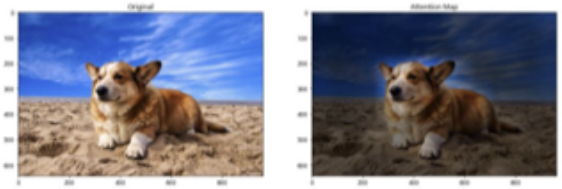
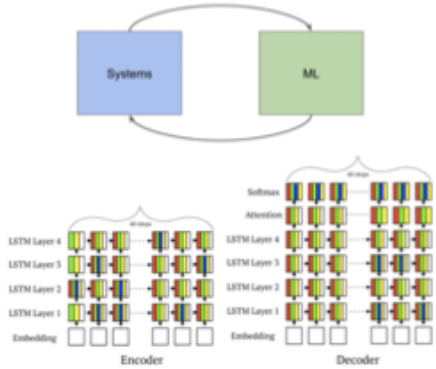


Image Classification

[Dosovitskiy et al. 2020]: Vision Transformer (ViT) outperforms ResNet-based baselines with substantially less compute.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** $\pm$ 0.04	87.76 $\pm$ 0.03	85.30 $\pm$ 0.02	87.54 $\pm$ 0.02	88.4/88.5*
ImageNet ReaL	**90.72** $\pm$ 0.05	90.54 $\pm$ 0.03	88.62 $\pm$ 0.05	90.54	90.55
CIFAR-10	**99.50** $\pm$ 0.06	99.42 $\pm$ 0.03	99.15 $\pm$ 0.03	99.37 $\pm$ 0.06	–
CIFAR-100	**94.55** $\pm$ 0.04	93.90 $\pm$ 0.05	93.25 $\pm$ 0.05	93.51 $\pm$ 0.08	–
Oxford-IIIT Pets	**97.56** $\pm$ 0.03	97.32 $\pm$ 0.11	94.67 $\pm$ 0.15	96.62 $\pm$ 0.23	–
Oxford Flowers-102	99.68 $\pm$ 0.02	**99.74** $\pm$ 0.00	99.61 $\pm$ 0.02	99.63 $\pm$ 0.03	–
VTAB (19 tasks)	**77.63** $\pm$ 0.23	76.28 $\pm$ 0.46	72.72 $\pm$ 0.21	76.29 $\pm$ 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



ML for Systems

[Zhou et al. 2020]: A Transformer-based compiler model (GO-one) speeds up a Transformer model!

Model (#devices)	GO-one (s)	HP (s)	METIS (s)	HDP (s)	Run-time speed up over HP / HDP	Search speed up over HDP
2-layer RNNLM (2)	0.173	0.192	0.355	0.191	9.9% / 9.4%	2.95x
4-layer RNNLM (4)	0.210	0.239	0.503	0.251	13.8% / 16.3%	1.76x
8-layer RNNLM (8)	0.320	0.332	OOM	0.764	3.8% / 58.1%	27.8x
2-layer GNMT (2)	0.301	0.384	0.344	0.327	27.6% / 14.3%	50x
4-layer GNMT (4)	0.350	0.469	0.486	0.432	34% / 23.4%	58.8x
8-layer GNMT (8)	0.440	0.562	OOM	0.693	21.7% / 36.5%	7.35x
2-layer Transformer-XL (2)	0.223	0.268	0.37	0.262	20.1% / 17.4%	40x
4-layer Transformer-XL (4)	0.230	0.27	OOM	0.259	17.4% / 12.6%	26.7x
8-layer Transformer-XL (8)	0.350	0.46	OOM	0.425	23.9% / 16.7%	16.7x
[illegible]	0.229	0.312	OOM	0.301	26.6% / 23.9%	13.5x
Inception (2) b64	0.423	0.731	OOM	0.498	42.1% / 29.3%	21.0x
AmoebaNet (4)	0.394	0.44	0.426	0.418	26.1% / 6.1%	58.8x
2-stack 18-layer WaveNet (2)	0.317	0.376	OOM	0.354	18.6% / 11.7%	6.67x
4-stack 36-layer WaveNet (4)	0.659	0.988	OOM	0.721	50% / 9.4%	20x
GEOMEAN	-	-	-	-	**26.0% / 18.2%**	**15x**

Many variants of Transformers:

Vision Transformer

Audio Transformer

https://web.stanford.edu/class/cs224n/

Recap: Attention is a general technique

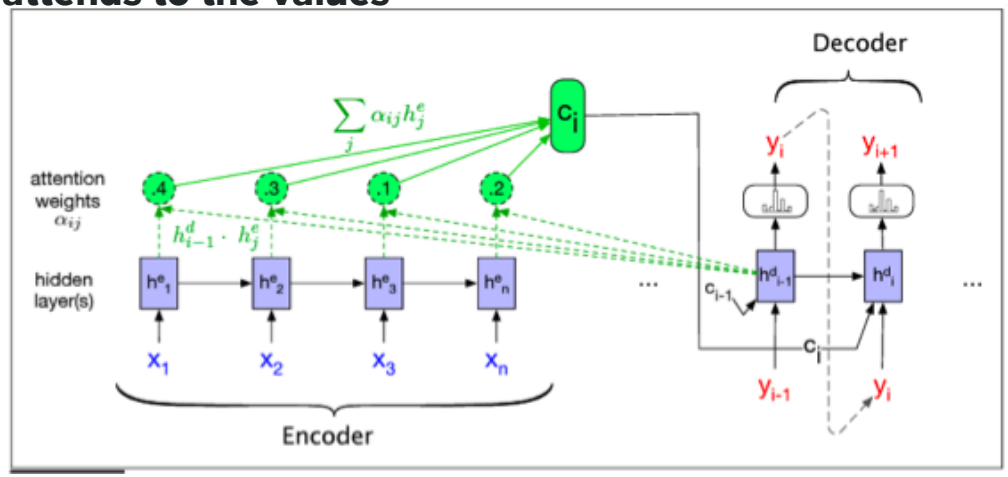
In the seq2seq+attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values)

We can use attention in many architectures (not just seq2seq) and many tasks (not just MT)

More general definition of attention:

➔ Given a set of vector **values**, and a vector **query**, **<u>attention</u>** is a technique to compute **a weighted sum of the values, dependent on the query**

We say that the **query attends to the values**



Attention is a general deep learning technique

More general definition of attention:

➔ Given a set of vector **values**, and a vector **query**, **<u>attention</u>** is a technique to compute a weighted sum of the values, dependent on the query

Intuition:

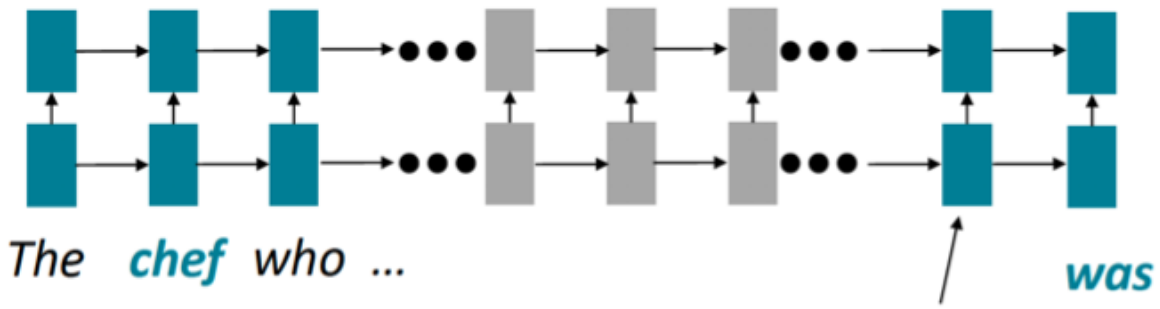
- The weighted sum is a **selective summary** of the information contain in the values, where the query determines which values to focus on
- Attention is a way to obtain a **fixed-size representation of an arbitrary set of representations**, *dependent on some other representation*

This is why attention and Transformers can generalize to not only text but many other modalities such as image representations

Issues with recurrent models

O(sequence length) steps for distant word pairs

- Hard to learn long-distance dependencies (gradient problems!)
- Linear order of words is "baked in"; not the right way to think about sentences



Info of ***chef*** has gone through O(sequence length) many layers!

The chef who came from the northern parts of India was …

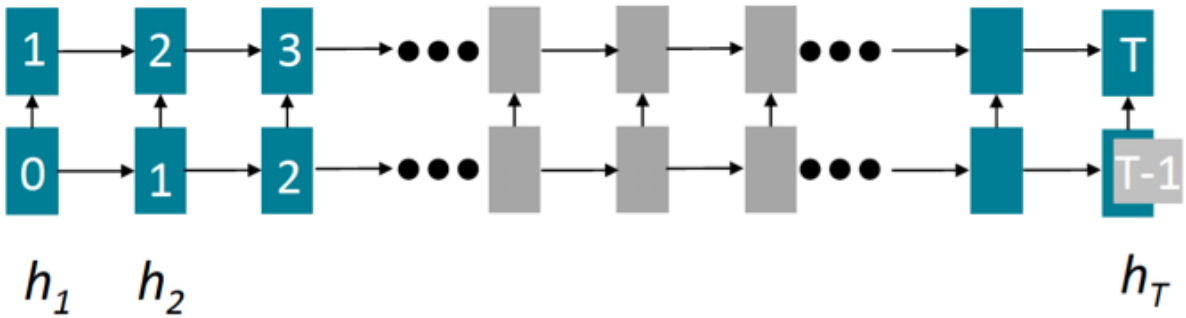
The interaction between "chef" and "was" has to pass through many hidden states

https://web.stanford.edu/class/cs224n/

Issues with recurrent models

Lack of parellilizability

- Future RNN hidden states can't be computed in full before past RNN hidden states have been computed



Inhibits training on very large datasets!

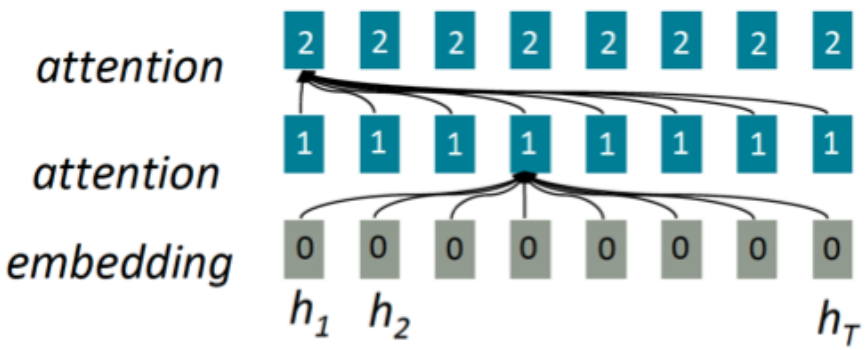
Numbers indicate min # of steps before a state can be computed

https://web.stanford.edu/class/cs224n/

If not recurrence, then what?

Attention

- Given a word as query, attention can be used to access and incorporate information from a set of values (other words)
- Can we do this within a single sentence? E.g., attention within the encoder itself
- All words can interact with each other and computation can be done in parallel!!



All words attend to all words in previous layer; most arrows here are omitted

You don't need the hidden states any more; attention is all you need (to compute representations).

https://web.stanford.edu/class/cs224n/

Transformer Encoder-Decoder

- Transformer is introduced as an **encoder-decoder architecture**;
  later we will see **encoder-only** & **decoder-only** transformers
- **Encoder** produces a sophisticated representation of the source sequence that the decoder will use to condition its generation process
- **Decoder** generates one token at the time to produce a target sequence; in the process, it produces representations that combine the history and a new token

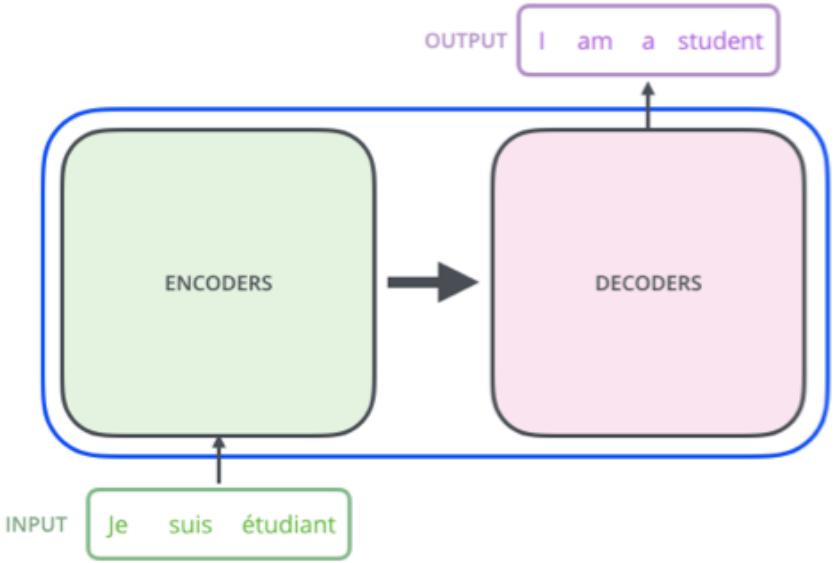
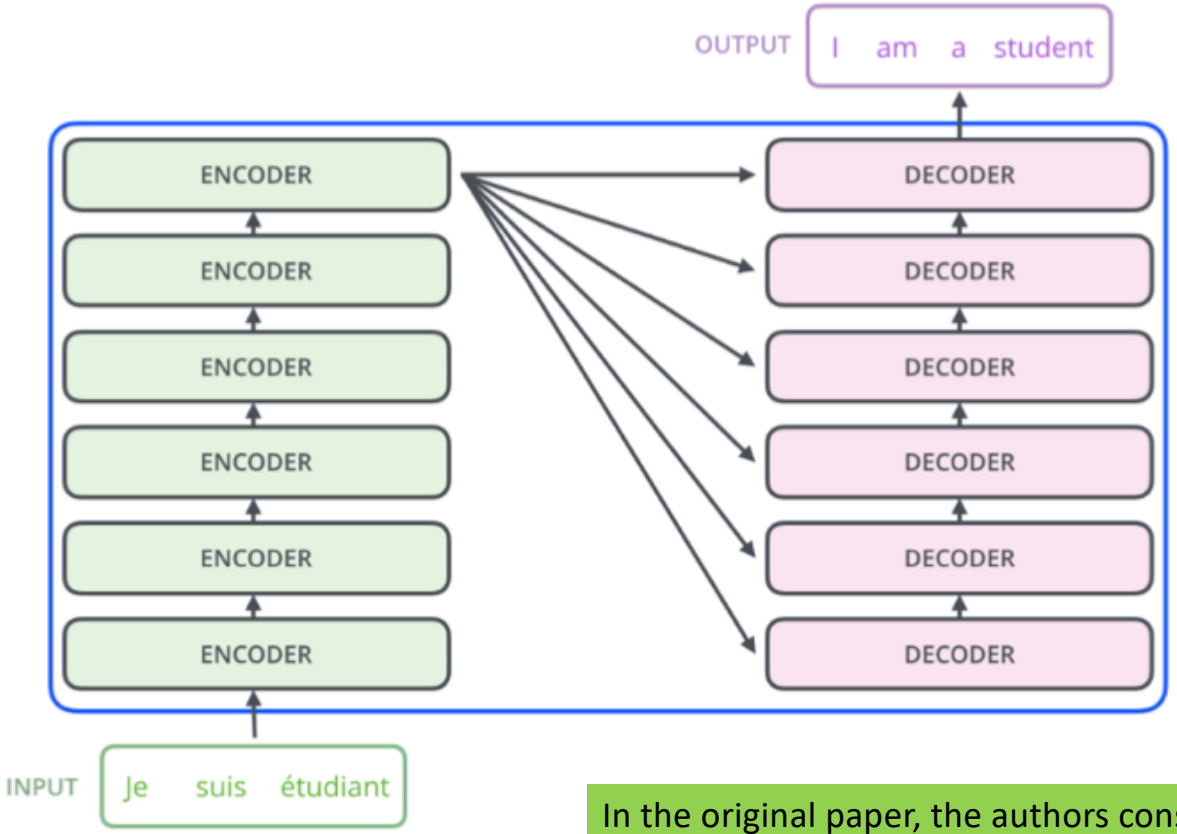


Figure: Jay Alammar

This high-level view is very similar to what we saw with seq2seq, but the underlying architecture is different (Transformers instead of RNNs)

A stack of encoder blocks



A stack of decoder blocks

In the original paper, the authors considered 6 Encoder blocks and 6 Decoder blocks (the number can be different)

Figure: [Jay Alammar]

Each encoder block consists of **self-attention** & **FFNN**

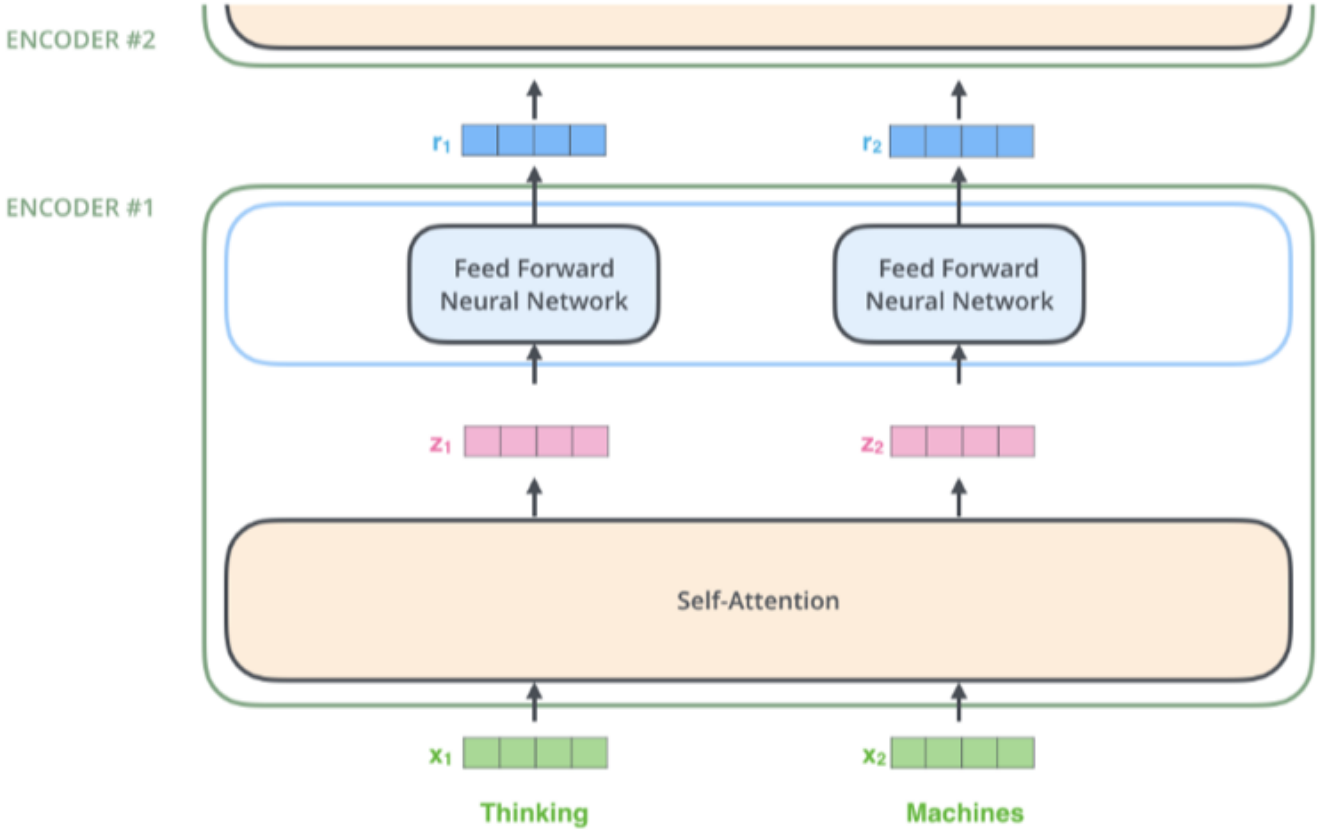


Figure: [Jay Alammar]

Deeper layers get outputs of the previous layers as inputs

Input to each encoder block has the same size as original token embeddings

This size is called "model dimensionality"

In the first layer, inputs are static token emeddings

Self-attention

Intuition for Self-Attention: 1

Problem with static embeddings (word2vec)

They are static! The embedding for a word doesn't reflect how its meaning changes in context.

`The chicken didn't cross the road because` $\boxed{\texttt{it}}$ `was too tired`

What is the meaning represented in the static embedding for "it"?

https://web.stanford.edu/~jurafsky/slp3/

Contextual Embeddings

`The chicken didn't cross the road because it`

What should be the properties of "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

What is “it” referring to? Either the chicken or the street

Co-reference resolution problem

https://web.stanford.edu/~jurafsky/slp3/

Contextual Embeddings

- Intuition: a representation of the meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings? **Attention**

https://web.stanford.edu/~jurafsky/slp3/

Intuition of attention

Build up the contextual embedding from a word by selectively integrating information from all the neighboring words

We say that a word "attends to" some neighboring words more than others

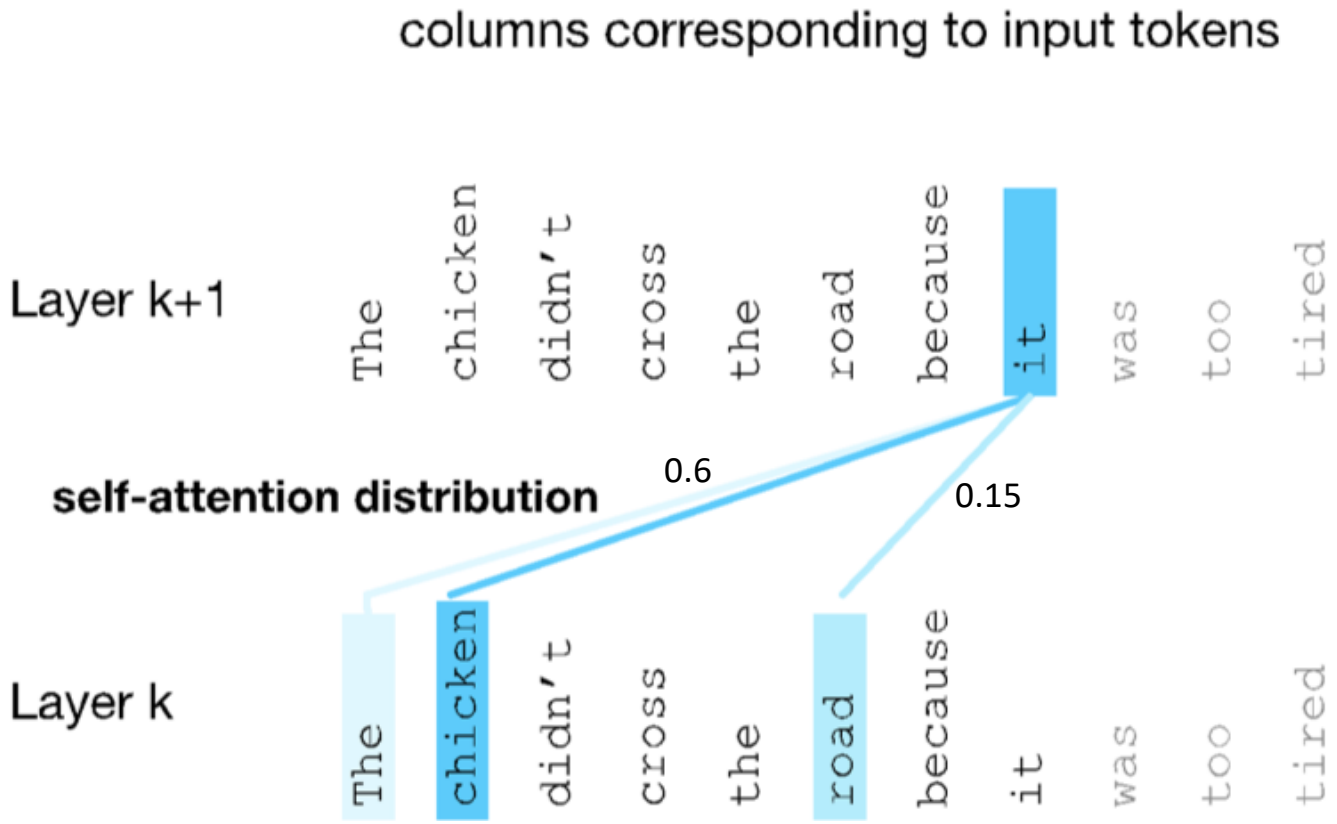
The **chicken** didn't cross the road because it was too **tired**

The **chicken** didn't cross the road because it was too **wide**

Can we have a mechanism to learn that
the representation of “it” should attend more to that of “chicken” in the first sentence and that of “road” in the second sentence?

https://web.stanford.edu/~jurafsky/slp3/

Intuition of attention



This image is a visualization given in a blog accompanying the Transformer paper.

Showed that the representation of "it" in a higher layer attends more to that of certain words in the lower layer.

https://web.stanford.edu/~jurafsky/slp3/

Simplified version of attention: a sum of prior words weighted by their similarity with the current word

Given a sequence of token embeddings:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produce: $\mathbf{a}_i$ = a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$)
Weighted by their similarity to $\mathbf{x}_i$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

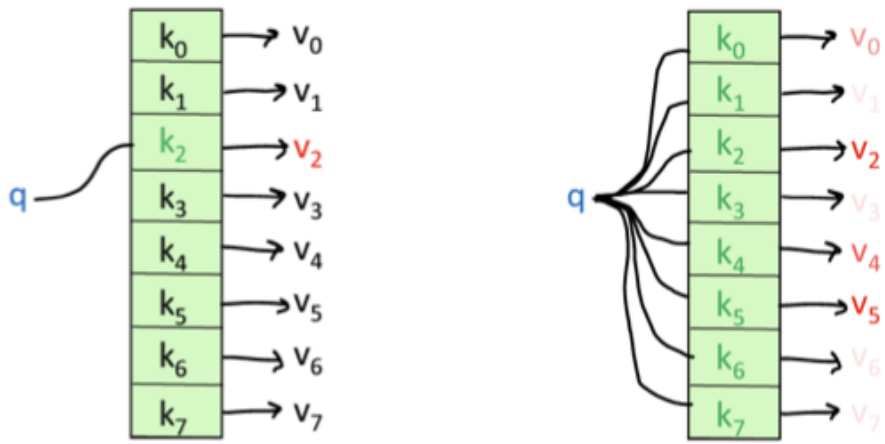
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

https://web.stanford.edu/~jurafsky/slp3/

Intuition for Self-attention: 2

Let's think of attention as a "fuzzy" or approximate hashtable:

- To look up a **value**, we compare a **query** against **keys** in a table.
- In a hashtable (shown on the bottom left):
  - Each **query** (hash) maps to exactly one **key**-**value** pair.
- In (self-)attention (shown on the bottom right):
  - Each **query** matches each **key** to varying degrees.
  - We return a sum of **values** weighted by the **query**-**key** match.



https://web.stanford.edu/class/cs224n/

An Actual Attention Head

High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector $\boldsymbol{x}_i$ plays:

- **query**: As *the current element* being compared to the preceding / other inputs.
- **key**: as *a preceding/other input* that is being compared to the current element to determine a similarity
- **value**: a value that gets weighted and summed

https://web.stanford.edu/~jurafsky/slp3/

An Actual Attention Head: slightly more complicated

We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:

- **query**: $\mathbf{W}^Q$
- **key**: $\mathbf{W}^K$
- **value**: $\mathbf{W}^V$

The three matrices consist of learnable parameters.

We expect the model to learn how to best project a vector into query, key and value.

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

https://web.stanford.edu/~jurafsky/slp3/

An Actual Attention Head: slightly more complicated

Given these 3 representation of $\boldsymbol{x}_i$

To compute similarity of current element $\boldsymbol{x}_i$ with some prior element $\boldsymbol{x}_j$
We'll use dot product between $\boldsymbol{q}_i$ and $\boldsymbol{k}_j$
And instead of summing up $\boldsymbol{x}_j$, we will sum up $\boldsymbol{v}_j$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathsf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathsf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathsf{V}}$$

https://web.stanford.edu/~jurafsky/slp3/

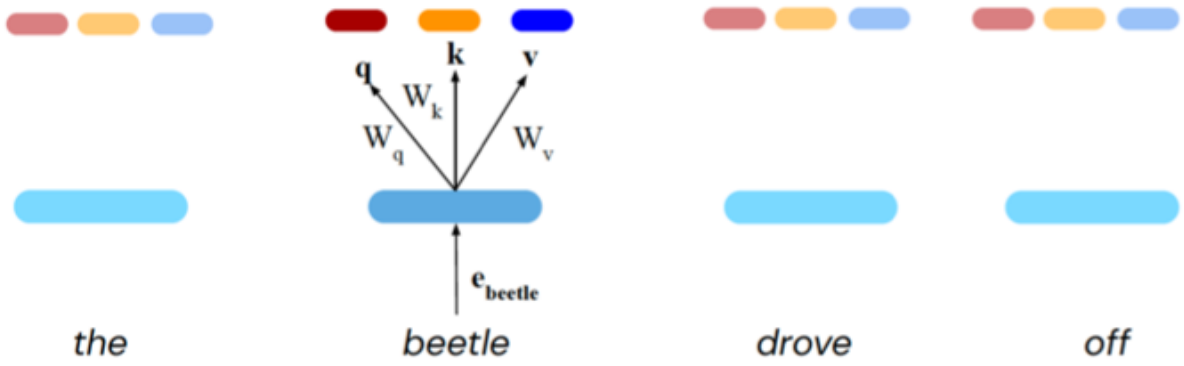
Transformers: Self-attention over input

$$\mathbf{q} = \mathbf{e}_{beetle}\, W_q$$

$$\mathbf{k} = \mathbf{e}_{beetle}\, W_k$$

$$\mathbf{V} = \mathbf{e}_{beetle}\, W_v$$

The same three matrices will be used for every word.
Hence the computations are independent of the length of the sequence.



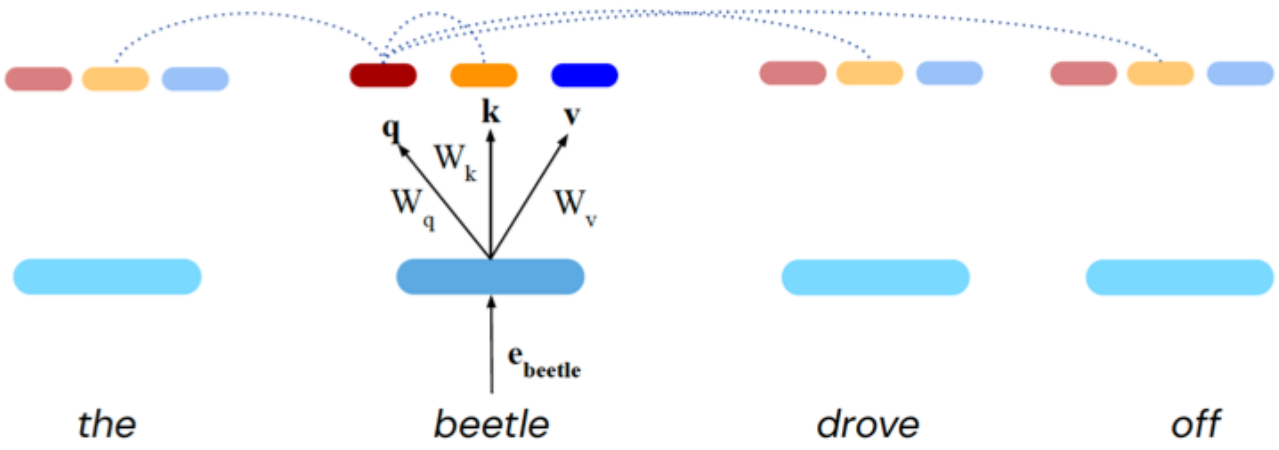
Query, Key, Value corresponding to each word

Word embeddings

Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Self-attention over input embeddings

Match q(beetle) with keys of all words including itself.



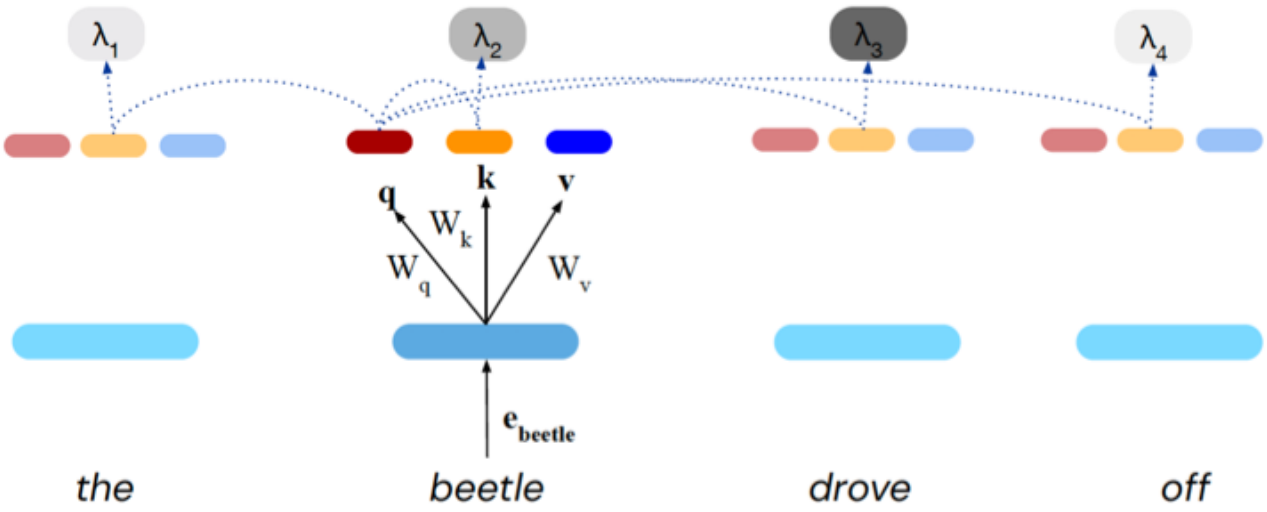
Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Self-attention over input embeddings

$$\lambda_i = \frac{e^{\mathbf{q}\cdot\mathbf{k_i}}}{\sum_{i=1}^{4} e^{\mathbf{q}\cdot\mathbf{k_i}}}$$

Match q(beetle) with keys of all words including itself.

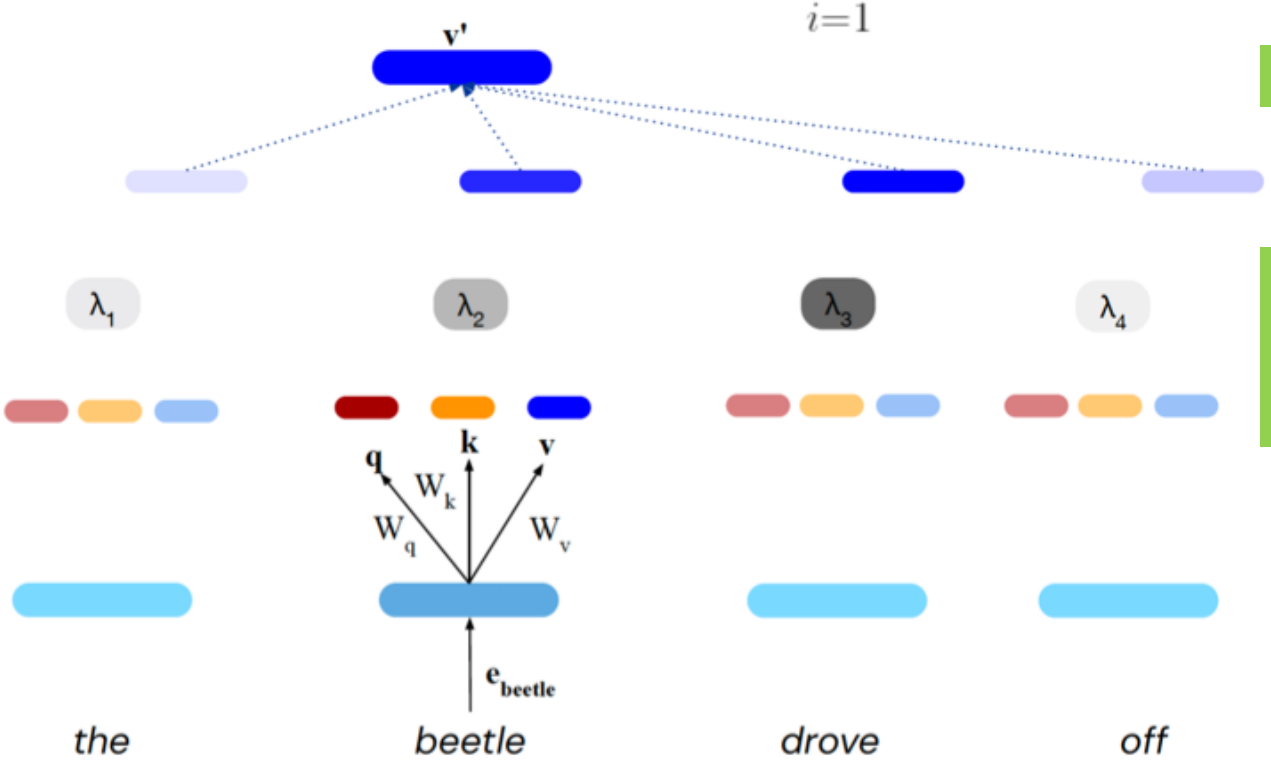
Softmax operation



Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Self-attention over input embeddings

$$\mathbf{v}' = \sum_{i=1}^{4} \lambda_i \mathbf{v}_i$$



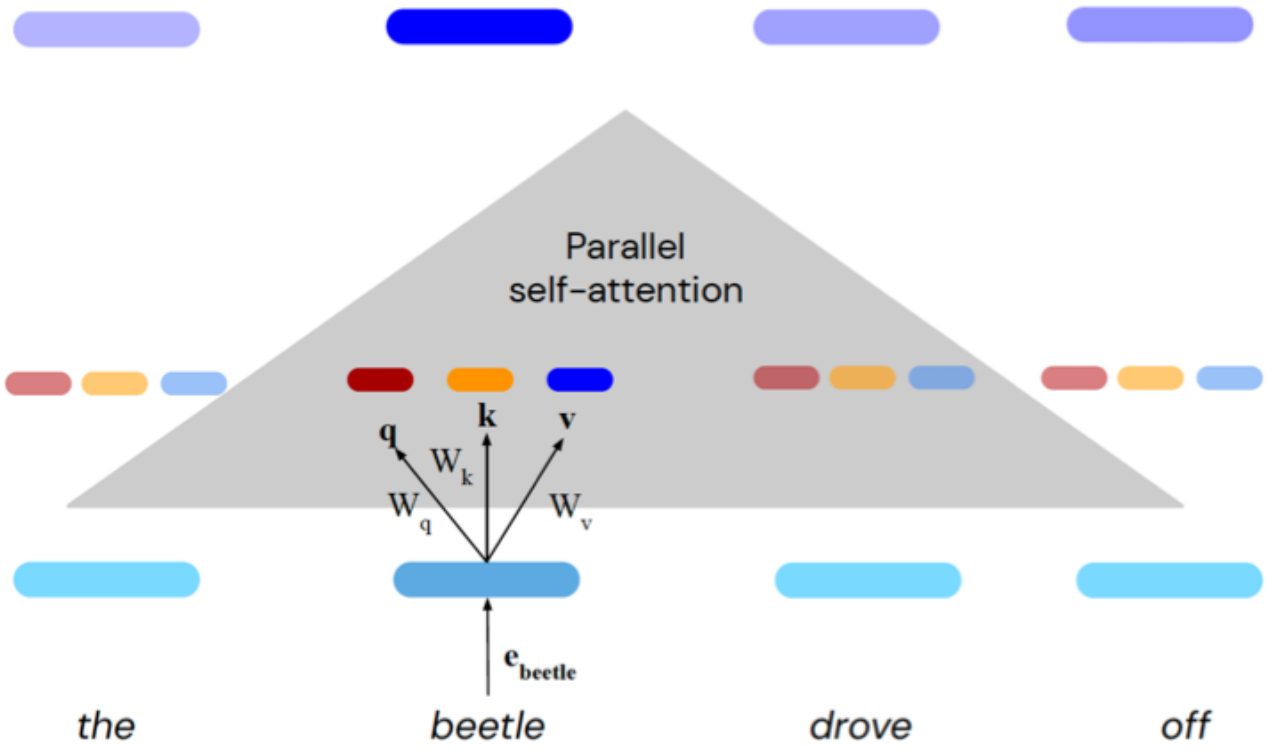
Match q(beetle) with keys of all words including itself.

Softmax operation

Take weighted average of the values of all words, to get the contextual representation of 'beetle'

Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Self-attention over all words (in parallel)

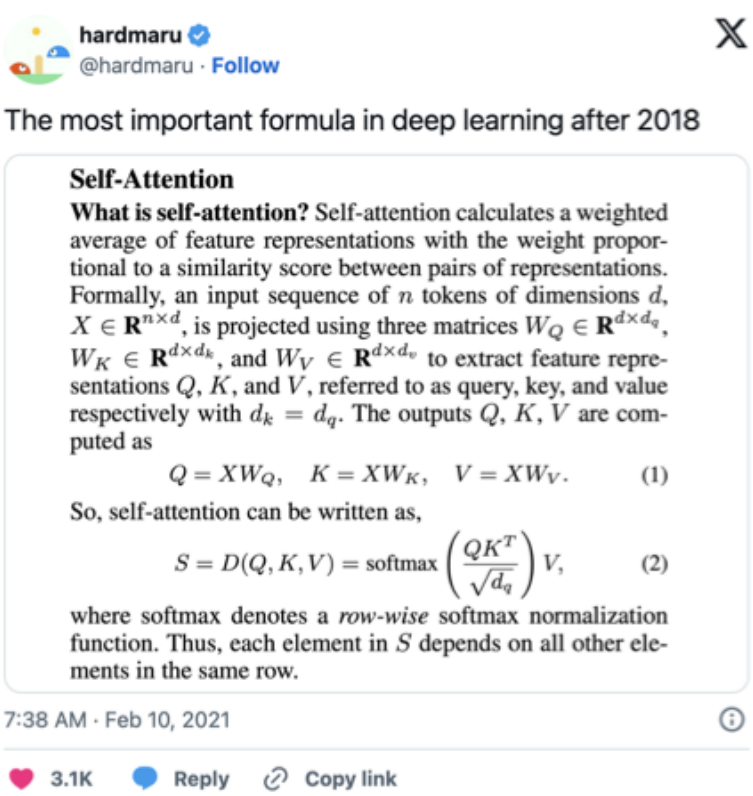


Two problems with RNN:
1. No connection between distant words
2. Sequential computation of representations

Both problems addressed by using Attention:
1. Each word can attend to all words in the sequence
2. Parallel computation of representations of all words possible

Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Self-attention: In equations



hardmaru
@hardmaru · Follow

The most important formula in deep learning after 2018

Self-Attention

What is self-attention? Self-attention calculates a weighted average of feature representations with the weight proportional to a similarity score between pairs of representations. Formally, an input sequence of n tokens of dimensions d, $X \in \mathbf{R}^{n \times d}$, is projected using three matrices $W_Q \in \mathbf{R}^{d \times d_q}$, $W_K \in \mathbf{R}^{d \times d_k}$, and $W_V \in \mathbf{R}^{d \times d_v}$ to extract feature representations Q, K, and V, referred to as query, key, and value respectively with $d_k = d_q$. The outputs Q, K, V are computed as

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \tag{1}$$

So, self-attention can be written as,

$$S = D(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_q}}\right) V, \tag{2}$$

where softmax denotes a *row-wise* softmax normalization function. Thus, each element in S depends on all other elements in the same row.

7:38 AM · Feb 10, 2021

3.1K Reply Copy link

x_i and x_j are two different words in the given sequence

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{V}}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

d_k is the dimension of the keys (a scaling constant)

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

$$\mathbf{a}_i = \sum \alpha_{ij} \mathbf{v}_j$$

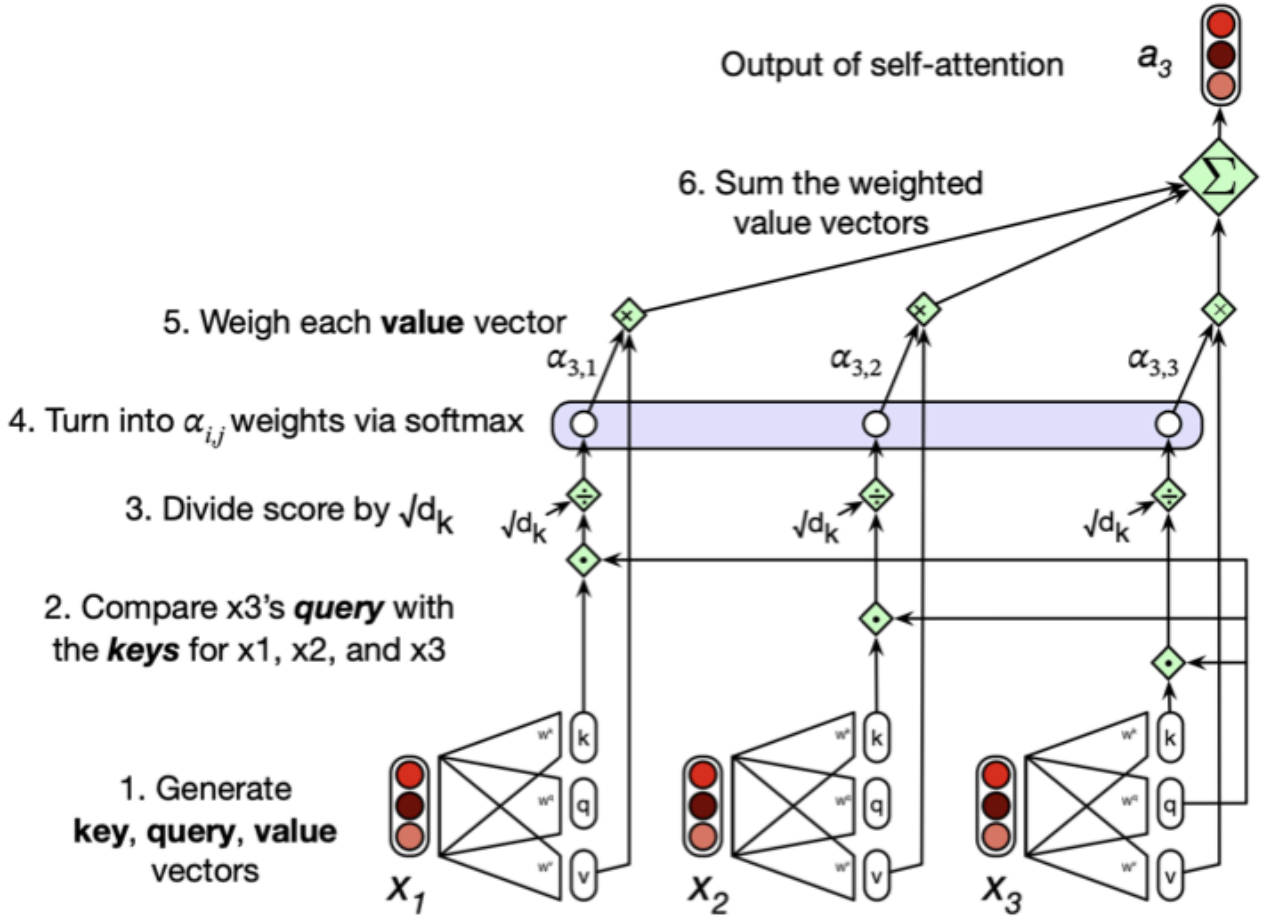
Attention that the i-th word is putting over the j-th word

Scaled dot-product: *more on this later*

Source: *https://theaisummer.com/self-attention/*

https://web.stanford.edu/~jurafsky/slp3/

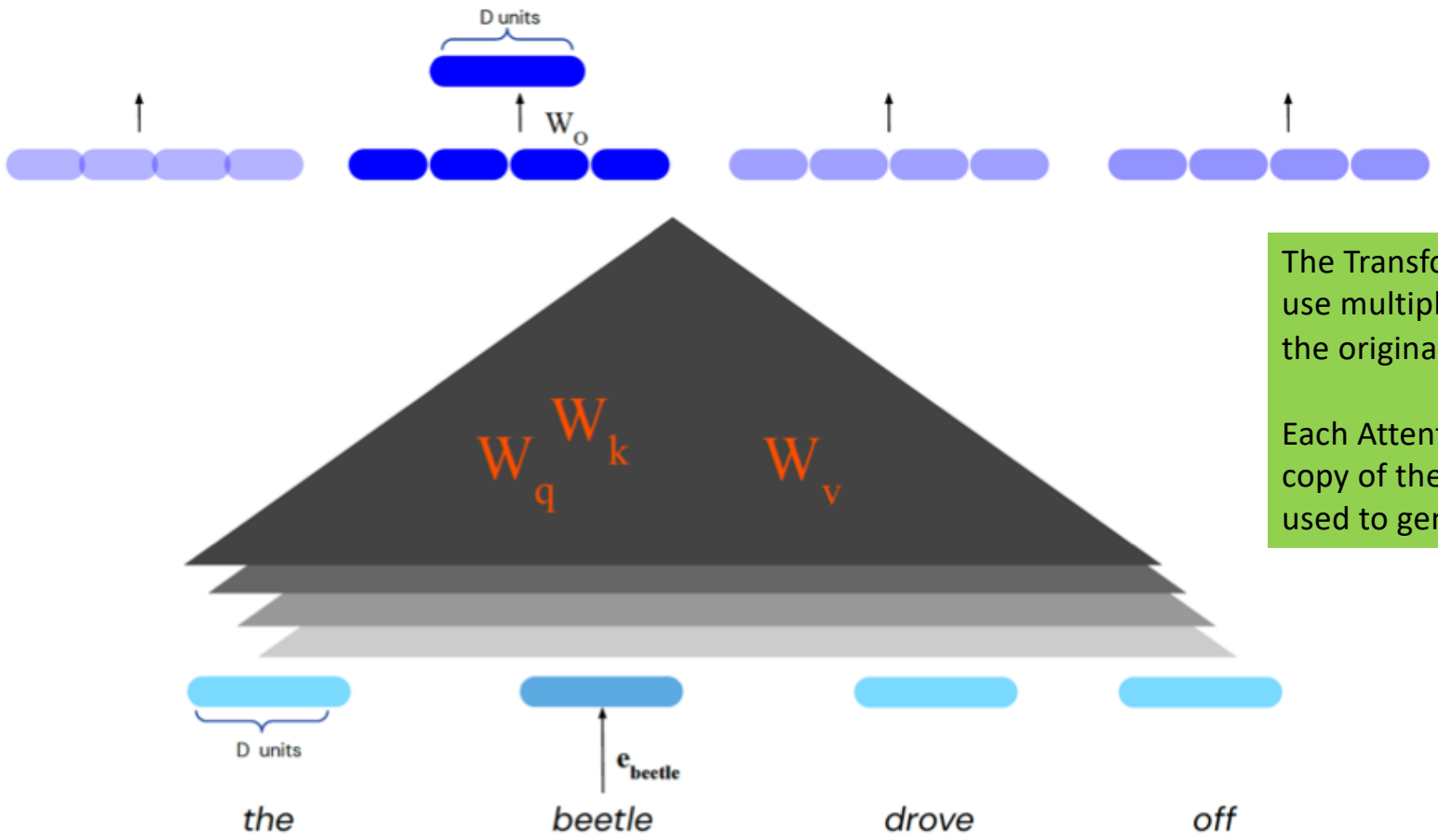
Calculating the self-attention output for x_3



https://web.stanford.edu/~jurafsky/slp3/

Multi-head attention

Multi-head Attention



The Transformer paper proposed to use multiple Attention heads (8 in the original paper).

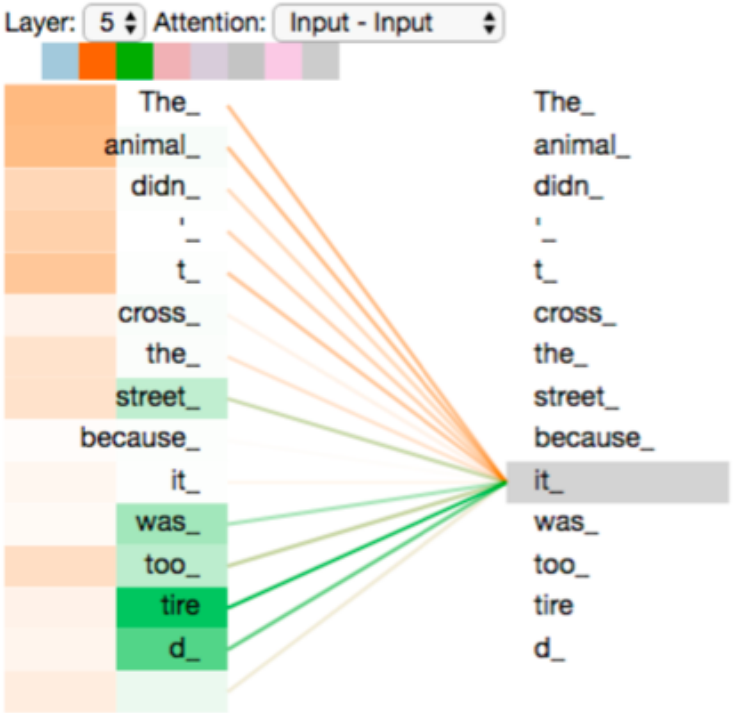
Each Attention head has its own copy of the 3 parameter matrices used to generate K, Q, V.

Source: *https://deepmind.com/learning-resources/deep-learning-lecture-series-2020*

Why Multi-head attention?

- What if we want to look in multiple places in the sentence at once?
  - For word i, we may want to focus on different j for different reasons
- We'll define multiple attention "heads" through multiple Q,K,V matrices
- Each head performs attention independently (parameters NOT shared)
- Then the outputs of all the heads are combined
- Each attention head gets to "look" at different things, and construct value vectors differently.

Multi-head attention: a simplified visualization



As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

Why Multi-head attention?

Prior work identified three important types of heads by looking at attention matrices

[Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned: https://arxiv.org/abs/1905.09418]

1. **Positional** heads that attend mostly to their neighboring words.
2. **Syntactic** heads that point to tokens with a specific syntactic relation.
3. Heads that point to **rare words** in the sentence (rare words usually important for understanding the context of all other words in the sentence).

Source: https://theaisummer.com/self-attention/

Multi-Head Attention: In Equations

- Each head might be attending to the context for different purposes

Computing the representation of x_i using h nos. of attention heads

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Qc}}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{Kc}}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{Vc}}; \quad \forall c \quad 1 \leq c \leq h$$

For the c-th attention head

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))$$

$$\mathbf{head}_i^c = \sum \alpha_{ij}^c \mathbf{v}_j^c$$

Output from the c-th head

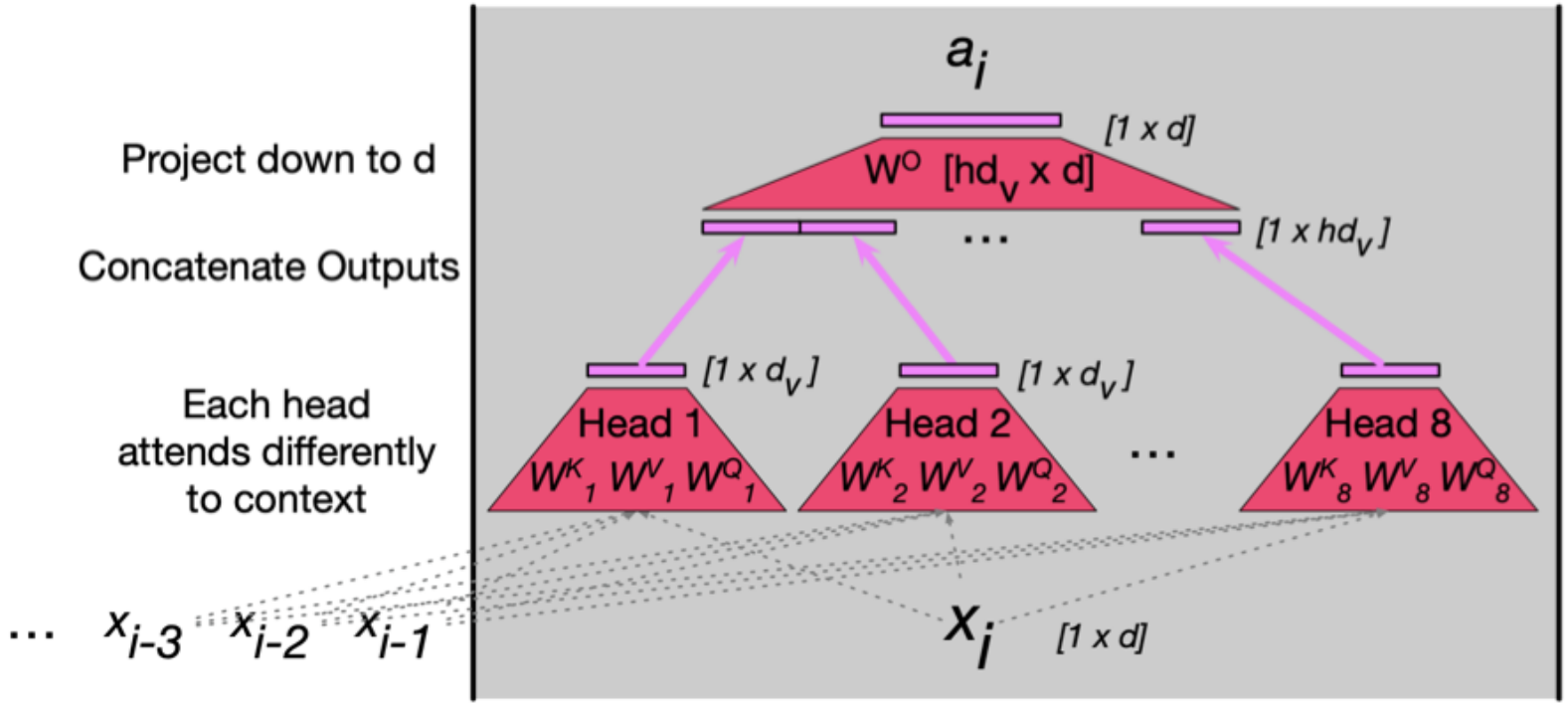
$$\mathbf{a}_i = (\mathbf{head}^1 \oplus \mathbf{head}^2 \ldots \oplus \mathbf{head}^h)\mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) = \mathbf{a}_i$$

Concatenate the output from all the h heads
Multiply the concatenation with a learned matrix so that a_i is of the same dimension as x_i

https://web.stanford.edu/~jurafsky/slp3/

Multi-head attention: another visualization



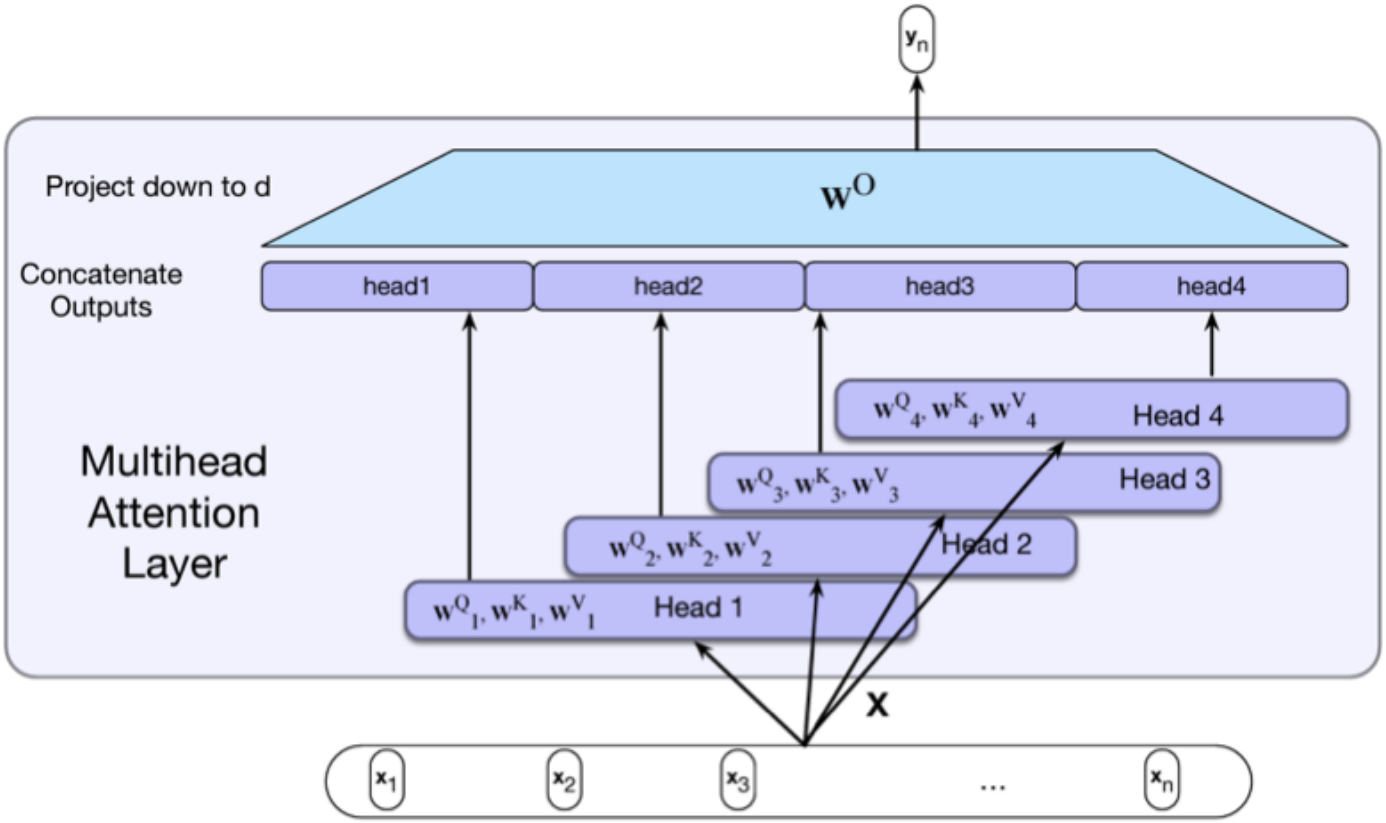
Output of each individual head is of the same dimension as the values

Outputs of 8 heads concatenated, and then projected back to the same dimension as the original embedding

All attention heads can carry out computations in parallel.

https://web.stanford.edu/~jurafsky/slp3/

Multi-head Attention Layer



https://web.stanford.edu/~jurafsky/slp3/

More on dimensions

Model dimension: d (=512)
Query, Key, Value dimensions: d_q, d_k, d_v (=64 each)

Projection matrices: $d \times d_k$ (= 512 x 64 each)

The output at each head: d_v
For "h" (=8) multi-heads: hd_v

To project it back to model dimension: W^O: $d \times hd_v$